Hamilton Cycles and Paths in the Noncrossing Partition Refinement Graph

Anonymous

Abstract

Let $\mathrm{NC}_R(n)$ be the cover graph of the refinement order on the noncrossing partitions of an n-element cyclically ordered set. Thus an edge merges two blocks, or reverses such a merge by splitting one block. We determine exactly when this graph has a Hamilton cycle and when it has a Hamilton path. If a one-vertex graph is regarded as Hamiltonian via its constant closed spanning walk, the cycle classification for all natural n is

$$\mathrm{NC}_R(n) \text{ is Hamiltonian} \iff n \in \{0,1\} \text{ or } (n \text{ is even and } n \geqslant 4).$$

With the analogous convention for a one-vertex path, $\mathrm{NC}_R(n)$ has a Hamilton path exactly when $n \leqslant 3$ or n is even. For odd $n = 2m+1$, block-count parity is a bipartition, and a signed Dyck-tree recurrence gives color-class difference $(-1)^{m+1}\mathrm{Cat}_m$. This rules out Hamilton cycles for odd $n \geqslant 3$ and Hamilton paths for odd $n \geqslant 5$; the remaining small orders are handled directly. For every even $n \geqslant 4$, we give a uniform Hamilton-cycle construction. A noncrossing partition is encoded bijectively by a noncrossing partial matching Q and a Boolean mask on the unmatched nonroot points. Each fixed Q is an odd-dimensional hypercube with a cyclic reflected Gray code. Deleting the lexicographically first chord makes the matchings into a rooted tree. Explicit refinement diamonds and an injective port assignment allow recursive square switching to join all Boolean cycles into one Hamilton cycle.

Mathematics Subject Classifications: 05A18, 05C45, 05C85

1 Introduction

Write $X_n = \{0, 1, \ldots, n-1\}$ with its cyclic order. A partition of X_n is *noncrossing* if there are no $a < b < c < d$ for which a, c belong to one block and b, d belong to another. The set $\mathrm{NC}(n)$ of all noncrossing partitions is the Kreweras lattice under refinement [4, 8, 9].

Refinement graph. Its cover graph $\mathrm{NC}_R(n)$ has vertex set $\mathrm{NC}(n)$; two vertices are adjacent when one is obtained from the other by merging exactly two blocks, with the result still noncrossing.

Anonymous Institution.

Why these are precisely the cover edges. The Kreweras lattice is graded by

$$\rho(\pi) = n - b(\pi),$$

where $b(\pi)$ is the number of blocks. If σ is obtained from π by merging two blocks, then $\pi < \sigma$ and $\rho(\sigma) = \rho(\pi)+1$, so no lattice element can lie strictly between them. Conversely, a cover raises the rank by one. Since every block of a coarsening is a union of blocks of the finer partition, lowering the number of blocks by one can only replace two blocks by their union. Thus the merge/split adjacency used below is exactly the cover relation, rather than a larger auxiliary graph.

The adjacency here is not the element-transfer adjacency in the cyclic Gray code of Huemer, Hurtado, Noy, and Omaña-Pulido [2]. In their graph, one element is moved between blocks in a prescribed local way; in our graph, an edge changes the number of blocks by exactly one through a whole-block merge or split. The distinction is already visible at $n = 3$: their graph is Hamiltonian, whereas the five-vertex refinement cover graph is bipartite with unequal color classes and hence is not Hamiltonian. Thus the known cyclic Gray code does not settle the problem considered here. The general permutation-language results of Hartung–Hoang–Mütze–Williams and Hoang–Mütze [1, 3] provide Hamilton paths for many quotient graphs, but do not identify the refinement cover graph with such a quotient. Given a finite list L of distinct elements of $\mathrm{NC}(n)$, a refinement Gray code for L is an ordering of all entries of L, each used once, such that consecutive partitions are adjacent in $\mathrm{NC}_R(n)$. Merino, Namrata, and Williams [5] prove that deciding whether an arbitrary input list L admits such an ordering is NP-complete. Their arbitrary-list problem differs from the full-family input $L = \mathrm{NC}(n)$. Corollary 2 shows that this full-family input admits a refinement Gray code if and only if $n \leqslant 3$ or n is even. For even $n \geqslant 4$, the ordering may be chosen cyclically, meaning that its last and first entries are also adjacent. For broader background on combinatorial Gray codes and their Hamiltonian formulations, see Mütze's survey [6].

Petersen [7] constructed a symmetric Boolean decomposition of the noncrossing partition lattice using valley hopping. Here the corresponding coordinates are described as follows: a Boolean block is indexed by a canonical noncrossing partial matching, and its Boolean coordinate is an explicit mask. The decomposition alone does not provide a Hamilton cycle: one must choose a different Gray edge for every child block, prove that the choices do not collide as directed edge occurrences, and retain one additional edge for the parent. The construction below supplies this compatibility through three types of ports and a recursive edge-piece assembly.

Our main result is the following complete classification.

Theorem 1 (Complete classification)**.** *Regard a one-vertex graph as Hamiltonian via its constant closed spanning walk. Then, for every $n \in \mathbb{N}$,*

$$\mathrm{NC}_R(n) \text{ has a Hamilton cycle} \iff n = 0 \text{ or } n = 1 \text{ or } (n \text{ is even and } 4 \leqslant n).$$

Consequently, in the usual nondegenerate range $n \geqslant 3$, $\mathrm{NC}_R(n)$ has a Hamilton cycle if and only if n is even.

Corollary 2 (Hamilton paths)**.** *Regard a one-vertex graph as having a Hamilton path of length zero. Then, for every* $n \in \mathbb{N}$*,*

$$\mathrm{NC}_R(n) \text{ has a Hamilton path} \iff n \leqslant 3 \text{ or } n \text{ is even.}$$

Proof. For $n = 0, 1$ the graph has one vertex, and for $n = 2$ it is a single edge. For $n = 3$, there are three two-block partitions, each adjacent to both the one-block and three-block partitions, so alternating through these five vertices gives a Hamilton path. If $n \geqslant 4$ is even, Theorem 1 gives a Hamilton cycle, and deleting one edge gives a Hamilton path. Finally, let $n = 2m + 1 \geqslant 5$. By Theorem 4, the two block-parity classes differ in size by $\mathrm{Cat}_m \geqslant 2$. A path in a bipartite graph alternates between the two classes, so no Hamilton path can exist. □

Theorem 1 is proved in Sections 2 and 3–5, which establish its two directions.
The constructive proof has the following dependencies:

$$\begin{array}{c} \text{matching–mask bijection} \longrightarrow \text{Gray cycles in the Boolean blocks} \\ \downarrow \\ \text{rooted matching tree} \longrightarrow \text{explicit joining diamonds} \\ \downarrow \\ \text{injective directed ports} \longrightarrow \text{recursive subtree cycle} \\ \downarrow \\ \text{Hamilton cycle.} \end{array}$$

Each arrow is justified below.

2 The odd-order obstruction

Let $b(\pi)$ denote the number of blocks of π. Every cover changes $b(\pi)$ by one, hence block-count parity colors the graph properly.

Lemma 3. *The graph* $\mathrm{NC}_R(n)$ *is bipartite, with color classes*

$$E_n = \{\pi : b(\pi) \text{ is even}\}, \qquad O_n = \{\pi : b(\pi) \text{ is odd}\}.$$

Proof. A cover relation merges two blocks or splits one block into two. Thus the two endpoints have block counts differing by exactly one, and therefore have opposite parity. □

Define the signed count

$$D_n := \sum_{\pi \in \mathrm{NC}(n)} (-1)^{b(\pi)} = |E_n| - |O_n|.$$

Use the standard Dyck-word, or equivalently plane-binary-tree, correspondence for non-crossing partitions in which the number of blocks is the number of peaks [8, 9]. We count only nonempty tree nodes: the empty tree has size zero, trees of size n correspond to

Dyck words of semilength n, and hence to $\mathrm{NC}(n)$. Let $p(T)$ be the peak count. A tree with $n+1$ nodes has the unique form $T = \mathrm{node}(L, R)$ with $|L| + |R| = n$. If L is empty, then $p(T) = 1 + p(R)$, so these trees contribute $-D_n$. If L is nonempty, write $|L| = k+1$; then $p(T) = p(L) + p(R)$ and $|R| = n - 1 - k$. Summing over the possible subtree sizes therefore gives

$$D_{n+1} = -D_n + \sum_{k=0}^{n-1} D_{k+1} D_{n-1-k}, \qquad D_0 = 1, \quad D_1 = -1. \tag{1}$$

Theorem 4 (Signed count)**.** *For $m \geqslant 1$,*

$$D_{2m} = 0, \qquad D_{2m+1} = (-1)^{m+1}\mathrm{Cat}_m,$$

where $\mathrm{Cat}_m = \frac{1}{m+1}\binom{2m}{m}$.

Proof. We use simultaneous strong induction on m. In (1) for D_{2m}, the two indices in every product sum to $2m-1$. Hence one index is positive and even, except for the boundary term $D_{2m-1}D_0$. Every nonboundary term vanishes by induction, and the boundary term cancels the initial $-D_{2m-1}$. Thus $D_{2m} = 0$.

For D_{2m+1}, all terms having a positive even index vanish. The remaining terms have indices $2j+1$ and $2(m-1-j)+1$. By induction their product is

$$(-1)^{j+1}\mathrm{Cat}_j(-1)^{m-j}\mathrm{Cat}_{m-1-j} = (-1)^{m+1}\mathrm{Cat}_j\mathrm{Cat}_{m-1-j}.$$

The initial term is zero because $D_{2m} = 0$. Summing and using the Catalan recurrence $\mathrm{Cat}_m = \sum_{j=0}^{m-1} \mathrm{Cat}_j\mathrm{Cat}_{m-1-j}$ proves the formula. The initial values $D_0 = 1$ and $D_1 = -1$ start the induction. □

Theorem 5 (Odd-order obstruction)**.** *If $n \geqslant 3$ is odd, then $\mathrm{NC}_R(n)$ has no Hamilton cycle.*

Proof. Write $n = 2m+1$. Since $n \geqslant 3$, $m \geqslant 1$, and Theorem 4 gives $|E_n| - |O_n| = (-1)^{m+1}\mathrm{Cat}_m \neq 0$. Every cycle in a bipartite graph uses equally many vertices from its two color classes. A spanning cycle would therefore force $|E_n| = |O_n|$, a contradiction. □

3 Canonical matching–mask coordinates

We now assume $n > 0$. A *canonical matching* is a set Q of ordered chords (a, b) with

$$0 < a < b < n,$$

such that distinct chords have disjoint endpoints and no two chords cross. The chords may be nested. Let $\mathrm{end}(Q)$ be their endpoint set and define the free points

$$F(Q) = X_n \setminus (\{0\} \cup \mathrm{end}(Q)).$$

For a chord endpoint x, let $e_Q(x)$ be the left endpoint of its chord. For $x \in F(Q)$, let $a_Q(x)$ be the left endpoint of the innermost chord whose open interval contains x, or 0 if there is no such chord.

For a mask $A \subseteq F(Q)$ define

$$\kappa_{Q,A}(x) = \begin{cases} e_Q(x), & x \in \mathrm{end}(Q), \\ a_Q(x), & x \in A, \\ x, & \text{otherwise.} \end{cases} \tag{2}$$

Let $P_Q(A)$ be the partition into the fibers of $\kappa_{Q,A}$.

Lemma 6 (Key-fiber geometry)**.** *If two distinct points have the same decoded key, then that key is either* 0 *or the left endpoint of a chord of Q. A fiber based at a chord (a, b) is contained in $[a, b]$. A point in that fiber cannot lie strictly inside a properly nested chord with a different left endpoint, and a point in the root fiber cannot lie strictly inside any chord of Q. Consequently, two distinct decoded fibers cannot cross.*

Proof. The first assertion follows directly from (2). A point can change its own key only by being a chord endpoint or by being selected in the mask. In the first case the common key is that chord's left endpoint. In the second it is the left endpoint of the innermost containing chord, or 0.

Fix $(a, b) \in Q$. Its endpoints decode to a. If another point x also decodes to a, then either x is one of these endpoints or x is selected and (a, b) is its innermost containing chord; in either case $a \leqslant x \leqslant b$. If (c, d) is properly nested in (a, b) and $c < x < d$, then (c, d) is a strictly inner containing chord, so the anchor of a selected x is at least $c > a$; hence x cannot decode to a. The same argument with no outer chord shows that a root-fiber point cannot lie inside any chord.

Suppose fibers based at r and s crossed, with $x_1 < y_1 < x_2 < y_2$ in the two fibers. If one key is 0, a point of the root fiber lies strictly between two points of the chord fiber and hence strictly inside its base chord, contradicting the root assertion. Suppose now that the two fibers are based at distinct chords $e = (a, b)$ and $f = (c, d)$. Since Q is noncrossing and endpoint-disjoint, their intervals are either disjoint or one is properly nested in the other. Disjoint intervals cannot contain alternating points. If f is nested in e, then the e-fiber point x_2, which lies between y_1 and y_2, lies strictly inside f, contradicting the nested-chord assertion for the e-fiber. If e is nested in f, the same argument uses the f-fiber point y_1 between x_1 and x_2. These cases exhaust the relative positions of the two base chords, so distinct fibers cannot cross. □

Theorem 7 (Boolean decomposition)**.** *For every $n > 0$, the map*

$$(Q, A) \longmapsto P_Q(A), \qquad Q \text{ canonical}, \quad A \subseteq F(Q),$$

is a bijection onto $\mathrm{NC}(n)$*. Moreover, if $q \in F(Q) \setminus A$, then $P_Q(A \cup \{q\})$ covers $P_Q(A)$ in the refinement lattice.*

Proof. First note that $\kappa_{Q,A}(x) \leqslant x$. By Lemma 6, four alternating points cannot lie in two different fibers. Thus $P_Q(A)$ is noncrossing.

Conversely, for $\pi \in \mathrm{NC}(n)$, take from every nonsingleton block not based at 0 the chord joining its minimum and maximum. Distinct blocks give endpoint-disjoint noncrossing chords, so this is a canonical matching $Q(\pi)$. Put in $A(\pi)$ exactly those remaining nonroot, nonendpoint points whose block is nontrivial.

We justify the anchor used by the decoder. Let x be such a remaining point, and let B be its block. If $0 \in B$, then no chord belonging to a different block can contain x in its open interval: its two endpoints, together with 0 and x, would give alternating points in two blocks. Hence $a_{Q(\pi)}(x) = 0 = \min B$. If $0 \notin B$, the chord $(\min B, \max B)$ belongs to $Q(\pi)$ and contains x. Any chord properly inside it belongs to a block nested inside B and cannot contain a point of B; any chord outside it is not innermost. Thus the innermost chord containing x is the chord of B itself, and $a_{Q(\pi)}(x) = \min B$. Chord endpoints decode to the same minimum by definition, while singleton points retain their own key. Therefore

$$\kappa_{Q(\pi),A(\pi)}(x) = \min(\text{the block of } x)$$

for every x, and hence $P_{Q(\pi)}(A(\pi)) = \pi$.

Conversely, start from $P_Q(A)$. Lemma 6 says that the key of any nonroot nonsingleton fiber is the left endpoint a of a chord $(a, b) \in Q$. Both a and b lie in that fiber. Every other point with key a lies in $[a, b]$, so a and b are its minimum and maximum. A second chord cannot have the same left endpoint because chord endpoints are disjoint. Thus taking the extrema of all nonroot nonsingleton fibers recovers exactly the chords of Q, not merely a matching with the same fibers. Once Q is known, a free point belongs to A exactly when its decoded key is not itself, equivalently when it lies in a nonsingleton fiber. This recovers A. The two reconstruction procedures are inverse, proving both injectivity and surjectivity.

Finally, if $q \notin A$, then q is a singleton fiber because its key is q. Inserting q into the mask changes only this key, from q to $a_Q(q)$. It therefore merges the singleton $\{q\}$ with exactly the fiber based at $a_Q(q)$ and leaves every other fiber unchanged. The block count drops by one, so this is one refinement cover. □

The dimension of the block indexed by Q is

$$d(Q) := |F(Q)| = n - 1 - 2|Q|. \tag{3}$$

If n is even, then $d(Q)$ is positive and odd.

4 Gray cycles, the block tree, and ports

Write $d = d(Q)$ and $F(Q) = \{u_0 < u_1 < \cdots < u_{d-1}\}$. We use the binary-reflected Gray list with u_0 as the least significant coordinate. Thus, with $G_0 = (\varnothing)$,

$$G_d = G_{d-1} \parallel \big(\mathrm{rev}(G_{d-1}) + u_{d-1}\big),$$

where $+u_{d-1}$ inserts u_{d-1} into every mask. Equivalently, the recursive coordinate list is supplied in descending free-point order. Adjacent masks differ by one free point, so Theorem 7 sends every Gray edge to a refinement edge.

Lemma 8 (Block Gray cycle)**.** *For even n, every block $\{P_Q(A) : A \subseteq F(Q)\}$ has a cyclic Gray listing that visits each of its vertices once. For $d(Q) \geqslant 3$ it is a simple cycle; for $d(Q) = 1$ it is the two directed occurrences of the unique edge, to be consumed by the tree switch below.*

Proof. The displayed recursion lists every subset once, changes one coordinate at every step, and has closing pair $\{u_{d-1}\}, \varnothing$. Equation (3) makes the dimension positive. Decoding gives the asserted refinement edges and the bijection gives distinctness and coverage. □

The particular Gray edges needed as ports are recorded separately, since they are stronger than mere cyclicity.

Lemma 9 (Low-coordinate Gray edges)**.** *Assume n is even, so $d \geqslant 1$. For the reflected Gray list on $F(Q) = \{u_0 < \cdots < u_{d-1}\}$:*

1. *if $d \geqslant 1$ and $H \subseteq F(Q) \setminus \{u_0\}$, then H and $H \cup \{u_0\}$ are consecutive;*
2. *if $d \geqslant 2$ and $H \subseteq F(Q) \setminus \{u_0, u_1\}$, then $H \cup \{u_0\}$ and $H \cup \{u_0, u_1\}$ are consecutive; and*
3. *the closing edge is $\{u_{d-1}\}, \varnothing$.*

Proof. For the first assertion, the case $d = 1$ is the list $\varnothing, \{u_0\}$. For $d > 1$, split G_d into its two recursive halves. If $u_{d-1} \notin H$, the required pair occurs in the first copy of G_{d-1} by induction. If $u_{d-1} \in H$, remove u_{d-1} from both masks; the resulting pair is consecutive in G_{d-1}, and adding u_{d-1} and reversing the copy preserves adjacency. The second assertion has the same argument, with the explicit base list G_2 and the pair involving u_1. Finally, the recursion starts at $\varnothing$, while its last mask is the singleton containing the newest, hence greatest, coordinate u_{d-1}. □

If $Q \neq \varnothing$, let $p(Q)$ be obtained by deleting the lexicographically first chord of Q.

Lemma 10 (Canonical block tree)**.** *The map p defines a rooted tree on canonical matchings, with root the empty matching. Each parent step decreases chord count by one and increases free dimension by two. Every canonical matching lies in the root subtree.*

Proof. Deleting a chord preserves endpoint disjointness and noncrossingness. It decreases $|Q|$ by one, so iteration terminates at the unique chordless matching. The parent is uniquely determined by the lexicographically first chord. Conversely, every nonroot matching has exactly that parent. Thus the directed parent relation is acyclic, connected to the empty matching, and has unique parent at every nonroot vertex; it is a rooted tree. Formula (3) gives the dimension change. □

Thus C is a child of Q precisely when

$$C = Q \cup \{e\}, \qquad e \text{ is the lexicographically first chord of } C;$$

equivalently, deleting the first chord of C gives Q. In particular, not every admissible chord insertion into Q is a child: the inserted chord must precede every old chord of Q.

Let C be a child of Q, with its first chord inserted between the ith and jth free points of Q, $i < j$. The reflected Gray cycles admit the following explicit opposite edges. Indices in a mask refer to the increasing free-point order of the corresponding block.

(i,j)	parent edge in Q	child edge in C
$1 \leqslant i < j$	$\{i,j\}, \{0,i,j\}$	$\varnothing, \{0\}$
$i = 0,\ j \geqslant 2$	$\{0,j\}, \{0,1,j\}$	$\varnothing, \{0\}$
$(i,j) = (0,1)$	$\varnothing, \{d(Q)-1\}$	$\varnothing, \{d(C)-1\}$.

(4)

Lemma 11 (Explicit joining diamond)**.** *For every parent–child pair Q, C, the two displayed Gray edges in the appropriate row of* (4)*, together with two cross-block refinement edges, form a four-cycle in* $\mathrm{NC}_R(n)$*.*

Proof. Write the free points of the parent as $u_0 < u_1 < \cdots < u_{d(Q)-1}$, and let the deleted chord be (u_i, u_j). The free points of the child are the remaining u_r's, in the same order. If B is a child mask, let $\widehat{B}$ be its order-preserving lift to the parent coordinates. Except in the special third row, the parent mask paired with B is

$$M(B) = \widehat{B} \cup \{i, j\}. \tag{5}$$

The decoder gives a direct comparison. First, u_i and u_j have the same anchor in the parent. Indeed, an old chord contains u_i in its open interval if and only if it contains u_j: if it contained exactly one of them, its endpoints would alternate with the new chord (u_i, u_j), contrary to the assumption that the child matching is noncrossing. Their chains of old containing chords are therefore identical. Write their common innermost anchor as

$$r = a_Q(u_i) = a_Q(u_j).$$

In the child, u_i and u_j are endpoints of one matching chord and decode to u_i; in the parent, selecting both in $M(B)$ sends both to r.

Now consider $x \notin \{u_i, u_j\}$. Chords other than (u_i, u_j) occur in both matchings. If x lies outside the deleted chord, its chain of containing chords is unchanged. If x lies inside it, then (u_i, u_j) can be its innermost chord only when its child key is u_i; otherwise a strictly nested chord remains innermost in both matchings. The order-preserving lift keeps the selected/unselected status of every surviving free point. Thus deleting the chord performs exactly one relabelling of keys:

$$\kappa_{Q,M(B)}(x) = \begin{cases} r, & \kappa_{C,B}(x) = u_i, \\ \kappa_{C,B}(x), & \kappa_{C,B}(x) \neq u_i. \end{cases}$$

Here $r < u_i$, so the child fibers with keys r and u_i are distinct. The formula shows in both directions that every other key class is unchanged. Thus $P_Q(M(B))$ is obtained from $P_C(B)$ by replacing precisely those two fibers by their union, and the pair is joined by one cover edge.

We now verify that the four masks in each row of (4) are the ones just described and that the horizontal pairs are Gray edges.

If $i > 0$, the least child free point lifts to u_0. Hence

$$M(\varnothing) = \{i, j\}, \qquad M(\{0\}) = \{0, i, j\}.$$

By part 1 of Lemma 9, both the parent pair and the child pair $\varnothing, \{0\}$ are edges of their fixed Gray cycles. The cross covers are

$$P_C(\varnothing) \lessdot P_Q(\{i, j\}), \qquad P_C(\{0\}) \lessdot P_Q(\{0, i, j\});$$

these are the first row.

If $i = 0$ and $j \geqslant 2$, the least child free point lifts to u_1. Thus

$$M(\varnothing) = \{0, j\}, \qquad M(\{0\}) = \{0, 1, j\}.$$

By part 2 of Lemma 9, the parent pair is a Gray-cycle edge; by part 1, so is the child pair. The cross covers are

$$P_C(\varnothing) \lessdot P_Q(\{0, j\}), \qquad P_C(\{0\}) \lessdot P_Q(\{0, 1, j\});$$

these are the second row.

Finally suppose $(i, j) = (0, 1)$. Here the two selected horizontal edges are the closing Gray edges. The rightmost child free point lifts to the rightmost parent free point. At the empty mask the endpoints u_0, u_1 are singletons in the parent and one chord fiber in the child, so inserting the chord performs exactly their merge. At the rightmost singleton mask, the selected rightmost point has the same outer anchor before and after the insertion because it lies to the right of u_1; the only changed keys are again those of u_0, u_1. Hence the cross covers are

$$P_Q(\varnothing) \lessdot P_C(\varnothing), \qquad P_Q(\{d(Q) - 1\}) \lessdot P_C(\{d(C) - 1\}).$$

By part 3 of Lemma 9, the two horizontal pairs are the closing edges in their respective block cycles. In all three cases the four vertices are distinct across the two Boolean blocks, both horizontal pairs are fixed block-cycle edges, and both vertical pairs are single merges. They therefore form the required four-cycle. □

Represent a Gray edge by a *port key*, either (q, H), meaning "flip coordinate q above ambient lower mask H", or *closing*. Here H is the endpoint not containing q, so the other endpoint is $H \cup \{q\}$. The three rows of (4) give one incoming key in the parent and one outgoing key in the child.

Lemma 12 (Port injection)**.** *For a fixed block Q, distinct children are assigned to distinct directed edge occurrences of the Gray cycle of Q. If Q is nonroot, its outgoing edge occurrence toward $p(Q)$ is not assigned to any child.*

Proof. Record a nonclosing directed Gray occurrence by a pair (q, H), where q is the flipped ambient point and H is the lower ambient mask. Equality of two such keys forces equality of both q and H; the closing occurrence is a separate key.

Let C be a child of Q, with first chord $e = (u_i, u_j)$. The parent-side key supplied by (4) is

$$\begin{cases} (u_0, \{u_i, u_j\}), & i > 0, \\ (u_1, \{u_0, u_j\}), & i = 0,\ j \geqslant 2, \\ \text{closing}, & (i, j) = (0, 1). \end{cases}$$

In each of the first two cases the lower mask is exactly the two-element endpoint set of e. Because the endpoints are ordered increasingly, that set uniquely recovers the ordered chord. The child itself is then recovered as $Q \cup \{e\}$. Thus two equal coordinate keys come from the same child. If both keys are closing, both first chords join the first two free points of Q, so again the chords and children coincide. A coordinate key cannot equal the closing key. This proves injectivity.

It remains to check that the edge retained for the parent of Q is not used by a child of Q. If the outgoing key is a coordinate key, its lower mask is $\varnothing$, whereas every incoming coordinate key has lower mask $\{u_i, u_j\} \neq \varnothing$; an incoming closing key has a different key type. If the outgoing key is closing, the first chord of Q joins its first two points that are free in $p(Q)$. Write this chord as $e = (a, b)$. If a child of Q existed, let $f = (c, d)$ be the chord whose deletion returns that child to Q. Both endpoints of f are free in Q, and all such points lie to the right of e, because e used the first two free points of $p(Q)$. Hence $b < c$, so $e <_{\text{lex}} f$. On the other hand, e remains a chord of the child while f must be its lexicographically first chord, forcing $f \leqslant_{\text{lex}} e$, a contradiction. Thus in this special case Q has no children at all. The outgoing occurrence is therefore unassigned in every case. □

The case $n = 4$. All three rows of (4) occur when $n = 4$. For the root matching $Q = \varnothing$, the free points are $u_0 = 1, u_1 = 2, u_2 = 3$, so its Boolean block is the 3-cube with eight vertices. The only other canonical matchings are the three one-chord matchings; each has one free point and hence a two-vertex Boolean block. The four blocks have sizes

$$2^3 + 2^1 + 2^1 + 2^1 = 14 = \text{Cat}_4.$$

Their ports in the root Gray cycle are listed below. Chord labels are ambient points of X_4, whereas the sets in the last two columns are mask-coordinate indices.

child chord	(i, j)	root edge	child edge
$(2, 3)$	$(1, 2)$	$\{1, 2\}, \{0, 1, 2\}$	$\varnothing, \{0\}$
$(1, 3)$	$(0, 2)$	$\{0, 2\}, \{0, 1, 2\}$	$\varnothing, \{0\}$
$(1, 2)$	$(0, 1)$	$\varnothing, \{2\}$	$\varnothing, \{0\}$.

The first two root edges share the vertex $\{0, 1, 2\}$, but they are distinct directed edge occurrences. The recursive construction below is indexed by directed occurrences, not by pairwise vertex-disjoint edges: each occurrence is replaced by one path piece, and adjacent pieces hand off at the shared Gray vertex. Splicing the three two-vertex child codes into these three root occurrences produces one cyclic code on all fourteen vertices. This is the $n = 4$ instance of the general recursion.

5 Recursive square switching

We use the elementary square switch: if two disjoint cycles contain opposite edges ab and cd of a four-cycle $abdca$, deleting ab, cd and adding ac, bd produces one cycle on the union of their vertices. Repeated switches modify the parent cycle, so the local square picture alone does not specify which parent edges remain. The recursive operation is given by the following list-concatenation principle.

At the recursive level, a *cyclic code* is a cyclic list of distinct vertices with adjacent consecutive entries. A two-vertex code records the two directed occurrences of its single undirected edge; codes on at least three vertices are ordinary simple cycles. The two-vertex convention is used only inside the recursion. The assembled root code below has at least three vertices and therefore gives a simple Hamilton cycle.

Lemma 13 (Cyclic piece concatenation)**.** *Let $v_0, v_1, \ldots, v_{r-1}$ be cyclically indexed, and for each directed occurrence $e_k = (v_k, v_{k+1})$ let K_k be a finite vertex list. Suppose that*

1. *each K_k is nonempty, has no repeated vertex, and starts at v_k;*
2. *consecutive vertices inside K_k are adjacent;*
3. *the last vertex of K_k is adjacent to v_{k+1}; and*
4. *the lists K_k are pairwise vertex-disjoint.*

Then

$$K_0 \parallel K_1 \parallel \cdots \parallel K_{r-1}$$

is a cyclic code with support the disjoint union of the supports of the K_k.

Proof. Within each piece, adjacency is assumption 2. Between successive pieces, the last vertex of K_k is adjacent by assumption 3 to v_{k+1}, which is the first vertex of K_{k+1} by assumption 1; the same argument joins the last piece back to K_0. Each piece is internally repetition-free, and assumption 4 prevents repetitions between pieces. Flattening the pieces therefore gives the asserted cyclic code and support. □

Directed occurrences are essential here. If a child code has only two vertices $[a, b]$, its cyclic occurrences are (a, b) and (b, a). Cutting at (a, b) leaves the path list $[b, a]$ (and conversely for the other orientation), so both vertices still occur exactly once. We are cutting a cyclic list at a specified occurrence; the underlying simple graph still has only one edge.

Lemma 14 (Block-tree assembly)**.** *For every canonical matching Q and even n, there is a cyclic code whose support is exactly the union of all Boolean blocks in the subtree rooted at Q. If Q is nonroot, the cyclic code retains the outgoing port prescribed by the diamond joining Q to $p(Q)$.*

Proof. We prove simultaneously the following three assertions for the subtree at Q:

1. there is one cyclic code Z_Q, which is a simple cycle when its support has at least three vertices;
2. its support is exactly the union of the Boolean blocks in the subtree;
3. if Q is nonroot, the prescribed outgoing directed edge of the block cycle of Q is still an edge of Z_Q.

Induct on $d(Q) = |F(Q)|$. A child is obtained by adding one chord and hence has dimension $d(Q) - 2$, so this is a well-founded induction. Assume the three assertions for every child.

The minimum possible dimension is $d(Q) = 1$. Then no child exists, since a new chord would require two free endpoints. The two directed occurrences of the one-dimensional block code are both unassigned, their pieces are the two singleton source lists, and their concatenation is the required two-vertex cyclic code. This establishes the base case.

Write the cyclic Gray list of the block of Q as $v_0, v_1, \ldots, v_{r-1}$, with indices modulo r, and regard $e_k = (v_k, v_{k+1})$ as a directed edge occurrence. By Lemma 12, each child is assigned to a distinct e_k. The assigned occurrences need not be vertex-disjoint, as the $n = 4$ example shows; distinctness is exactly what is required because the construction builds one replacement piece for each source occurrence. For an unassigned occurrence e_k, define the corresponding piece to be the singleton list $[v_k]$. For an occurrence assigned to a child C, the inductive cyclic code Z_C still contains its prescribed outgoing edge ab. The joining diamond has opposite directed occurrences e_k and ab. Cutting Z_C at ab turns it into a path list through every vertex of the child subtree, in one of its two orientations. Orient it as $W_C = [w_0, \ldots, w_s]$ so that the diamond supplies the cross edges $v_k w_0$ and $w_s v_{k+1}$. The assigned piece is the list

$$K_k = [v_k] \parallel W_C.$$

Its first vertex is v_k, its internal adjacencies are the first cross edge followed by the cut child path, and its last vertex is adjacent through the second cross edge to v_{k+1}. Thus every piece associated with e_k starts at the source of e_k and hands off to the source of e_{k+1}; the preceding paragraph covers the two-vertex child case as well.

We verify that concatenation introduces neither omissions nor repetitions. The vertices v_k are distinct because the block Gray list is a cyclic listing. A child-subtree piece contains no vertex in the block of Q: the canonical matching of every one of its vertices lies at or below that child, hence strictly below Q in the matching tree, whereas the vertices v_k have canonical matching exactly Q. Two different child-subtree pieces are disjoint because a rooted tree has disjoint subtrees below distinct children. Finally, port injectivity ensures

that no child subtree is inserted twice. Therefore all pieces are nonempty, internally simple, and pairwise disjoint.

Their internal edges are either edges of a cut child code or the first cross edge in its joining diamond. The exit of the kth piece is adjacent through the second cross edge to v_{k+1}, the head of the next piece; the final piece hands off to v_0 in exactly the same way. The four hypotheses of Lemma 13 are now verified. Its concatenation is a single cyclic code, and a simple cycle whenever it has at least three vertices.

Every vertex in the block of Q occurs as exactly one v_k. Every proper descendant belongs to the subtree of a unique child, and that entire child subtree occurs in the unique piece assigned to that child. The support is therefore exactly the subtree of Q. If Q is nonroot, its outgoing occurrence is unassigned by Lemma 12; its piece is the singleton $[v_k]$, so the handoff edge $v_k v_{k+1}$ remains present in the flattened cycle. This proves the third assertion and closes the induction. □

Theorem 15 (Uniform even-order construction)**.** *For every even $n \geqslant 4$, $\mathrm{NC}_R(n)$ has a Hamilton cycle.*

Proof. Apply Lemma 14 to the empty matching. By Lemma 10, every canonical matching lies in its subtree. By Theorem 7, the corresponding Boolean blocks partition all of $\mathrm{NC}(n)$. The assembled root cycle is therefore spanning. Since $n \geqslant 4$, $|\mathrm{NC}(n)| = \mathrm{Cat}_n \geqslant 3$ [8], so the closed spanning walk is a Hamilton cycle in the simple graph. □

6 Proof of the classification

Proof of Theorem 1. For even $n \geqslant 4$, apply Theorem 15. For odd $n \geqslant 3$, apply Theorem 5.

It remains to inspect $n < 4$. There is one noncrossing partition for $n = 0$ and one for $n = 1$, so the convention in the theorem regards both singleton graphs as Hamiltonian. For $n = 2$, the refinement graph consists of one edge on two vertices and has no simple Hamilton cycle. For $n = 3$, the odd obstruction applies (equivalently, the five-vertex bipartition is unbalanced). These cases give exactly the stated disjunction. □

7 Concluding remarks

For odd orders, the two block-parity classes have different sizes. For even orders, every Boolean block has odd dimension, and the lexicographic chord tree provides injectively assigned joining ports. The even-order construction does not require a Hamilton path with prescribed endpoints inside each Boolean block. It reserves specified Gray edges and joins parent and child cycles through four-cycles in the refinement graph.

8 Machine-checked correspondence

The main Lean 4 import is `Hamilton.HamiltonianCycles`. The final cycle and path classifications are `NCRefinementGraph_fin_isHamiltonian_iff` and `NCRefinementGraph_`

`fin_isHamiltonianPath_iff`. To trace an intermediate result, locate its paper label or proof component in Table 1 and then search for the declarations in the second column. The table is a navigation index; the mathematical identification of the paper's definitions with their Lean counterparts is explained in the surrounding prose.

The first identification to check is graph adjacency. The declaration

`NCRefinementGraph_adj_iff_single_merge`

expands adjacency to a two-block merge in either direction. The declarations `mergesTo_numBlocks` and `mergesTo_le` state that such a merge removes one block and moves upward in the refinement order. Conversely, the graded-lattice argument in the introduction shows that a cover cannot skip a rank and therefore must merge exactly two blocks. Thus the Lean adjacency is the cover-graph adjacency used in the paper.

Table 1: Paper-to-Lean correspondence.

Paper statement or proof step	Lean declarations
def:refinement-graph	NCRefinementGraph_adj_iff_single_merge, mergesTo_numBlocks, mergesTo_le
lem:bipartite	NCRefinementGraph_isBipartite
thm:signed-count	signedDyckSumTree_closed_form, signedNCCount_univ_eq_signedDyckSumTree
thm:odd-obstruction	NCRefinementGraph_fin_odd_geq3_not_isHamiltonian
lem:key-fibers	commonKey_root_or_chord, point_in_chord_hull, point_avoids_nested_chord, root_point_avoids_chord, decodedFinpartition_isNoncrossing
thm:boolean-decomposition	decodeState_bijective, stateEquivNC, decodedNC_mergesTo_insert
lem:block-gray-cycle	rankGrayCycle, rankGrayVertices_nodup, rankGrayVertices_chain
lem:gray-ports	coordinateZero_rankGray_consecutive, coordinateOne_rankGray_consecutive, rankGray_closing_flip
lem:canonical-block-tree	parent_card, parentOption_eq_none_iff, inSubtree_root
lem:joining-diamond	parent_decodeKey_eq_relabel_child, decodedNC_child_adj_parentMask, joiningDiamond_nonempty, joiningDiamond
lem:port-injection	portAssignedToPair_exists, assignedPair_injective, outgoingPair_not_assigned
lem:piece-concatenation	EdgePieceSystem.toCycleCode
lem:tree-assembly	edgePieces_pairwise_disjoint, subtreeEdgePieceSystem, subtreeCycle, assembledSubtreeCycleCode_support, assembledSubtreeCycleCode_outgoing, rootCycleCode_support
thm:uniform-even	NCRefinementGraph_fin_even_geq4_isHamiltonian_petersen
thm:classification	NCRefinementGraph_fin_isHamiltonian_iff

Paper statement or proof step	Lean declarations
`cor:hamilton-paths` ($n \leqslant 3$)	`NCRefinementGraph_fin_zero_isHamiltonianPath`, `NCRefinementGraph_fin_one_isHamiltonianPath`, `NCRefinementGraph_fin_two_isHamiltonianPath`, `NCRefinementGraph_fin_three_isHamiltonianPath`, `cardThree_hamiltonianPath_between_twoBlock`
`cor:hamilton-paths`: even case	`IsHamiltonian.isHamiltonianPath`
`cor:hamilton-paths`: odd case	`NCRefinementGraph_evenBlocks_oddBlocks_diff_at_most_one_of_isHamiltonianPath`, `NCRefinementGraph_fin_odd_geq5_not_isHamiltonianPath`
`cor:hamilton-paths`: final	`NCRefinementGraph_fin_isHamiltonianPath_iff`

From the `lean4` directory, run

```
lake build Hamilton.HamiltonianCycles
lake env lean --trust=0 Hamilton/HamiltonianCyclesTheoremMap.lean
lake env lean --trust=0 Hamilton/HamiltonianCyclesAxiomAudit.lean
```

The first command builds the formalization, the second checks the declarations in Table 1, and the third reports for the audited construction, obstruction, cycle-classification, and path-classification endpoints exactly

`[propext, Classical.choice, Quot.sound]`.

These are standard Lean and Mathlib logical dependencies. No additional axiom, `sorry`, `admit`, or `native_decide` occurs in the dependency closure of these theorems.

The audit checks the Lean theorems under their formal definitions and the foundational dependencies just listed. The correspondence between those definitions and the mathematical objects in the paper is part of the mathematical exposition.

The Lean development proves the merge/split adjacency and the cycle and path classifications for the full family NC(n). It does not formalize the arbitrary-list decision problem of Merino, Namrata, and Williams; the comparison above follows directly from their definition.

The kernel-checked formalization is publicly archived. The archive identifier is omitted from this anonymous copy.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the author used OpenAI Codex for exploratory proof development, drafting and copy-editing, code development, and the Lean 4 formalization. Claude (Anthropic) was used during the initial exploration. The author checked the mathematical claims and citations, compared the manuscript statements with the Lean declarations, and reran the formal build and axiom audit. AI output is not treated as mathematical evidence. The author is responsible for the definitions, proofs, citations, code, and manuscript.

References

[1] E. Hartung, H. P. Hoang, T. Mütze, and A. Williams, Combinatorial generation via permutation languages. I. Fundamentals, *Trans. Amer. Math. Soc.* **375** (2022), 2255–2291; doi:10.1090/tran/8199; arXiv:1906.06069.

[2] C. Huemer, F. Hurtado, M. Noy, and E. Omaña-Pulido, Gray codes for non-crossing partitions and dissections of a convex polygon, *Discrete Appl. Math.* **157** (2009), 1509–1520; doi:10.1016/j.dam.2008.06.018.

[3] H. P. Hoang and T. Mütze, Combinatorial generation via permutation languages. II. Lattice congruences, *Israel J. Math.* **244** (2021), 359–417; doi:10.1007/s11856-021-2186-1; arXiv:1911.12078.

[4] G. Kreweras, Sur les partitions non croisées d'un cycle, *Discrete Math.* **1** (1972), 333–350; doi:10.1016/0012-365X(72)90041-6.

[5] A. Merino, Namrata, and A. Williams, On the hardness of Gray code problems for combinatorial objects, preprint, 2024; arXiv:2401.14963.

[6] T. Mütze, Combinatorial Gray codes—an updated survey, *Electron. J. Combin.* **30**(3) (2023), DS26; doi:10.37236/11023; arXiv:2202.01280.

[7] T. K. Petersen, On the shard intersection order of a Coxeter group, *SIAM J. Discrete Math.* **27** (2013), 1880–1912; doi:10.1137/110847202; arXiv:1108.5761.

[8] R. Simion, Noncrossing partitions, *Discrete Math.* **217** (2000), 367–409; doi:10.1016/S0012-365X(99)00273-3.

[9] R. Simion and D. Ullman, On the structure of the lattice of noncrossing partitions, *Discrete Math.* **98** (1991), 193–206; doi:10.1016/0012-365X(91)90376-D.